†

| | |
|---|---|
| UFABC | **Universidade Federal do ABC**<br>**Bacharelado em Ciência da Computação**<br>**Disciplina:** Sistemas Operacionais<br>**Prof.:** VALÉRIO BATISTA<br>**Turma:** DA1 DA2 NA1 NA2 **Sala:** A-108/114-0<br>**Exame:** SO - Lista5 **Data:** 15-03-2024 |

**Ass.:** _____

**Estudante:** Val **ID/RA:** 12345678

#E580#V78548 - 2024-03-14 - 20:56:52

| | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| A | | | | | | | | | | | | |
| B | | | | | | | | | | | | |
| C | | | | | | | | | | | | |
| D | | | | | | | | | | | | |

**Instruções:**

(a) Favor repassar suas respostas neste link: https://forms.gle/FoxDHpbsZxSjjnYs8
(b) Formulário disponível até 22/3/24 às 23:59:59
(c) Somente a 1a. submissão será registrada (re-submissões serão invalidadas)

**Questões de Múltipla Escolha:**

1. Considere o modelo de multithreading com as seguintes características: requer gerenciamento pelo kernel, mas com isso aproveita a arquitetura multicore, mesmo que crie bem mais threads que o necessário; permite execução de outra thread enquanto uma delas faz chamada de E/S. Tal modelo é:
A. Um-para-um. B. Muitos-para-muitos. C. Um-para-muitos D. Muitos-para-um.

2. Considere as seguintes componentes de um processo: *C*ódigo de programa, *D*ados (var. globais, arquivos abertos), *H*eap, *P*rogram Counter, *R*egistradores e *S*tack (var. locais, parâm. de funções, ender. retorno). Se o processo for subdividido em threads, estas irão compartilhar
A. *C, D, H*, mas não *P, R, S*. B. *C, D*, mas não *H, P, R, S*. C. *C, D, S*, mas não *H, P, R*. D. *C, D, H, P*, mas não *R, S*.

3. O uso de threads é importante porque
A. Substituem processos longos por outros mais curtos. B. Criar novos processos tem alto custo computacional e exige muitos recursos. C. Sem ela não é possível rodar processos. D. É a única forma de utilizarmos processamento paralelo.

4. Os ítens abaixo exemplificam uso the threads, exceto:
A. Vídeo sendo exibido enquanto a próxima parte dele ainda está baixando. B. Dublador grava áudio enquanto assiste a vídeo original em língua estrangeira. C. Texto de mensagem sendo escrito enquanto o servidor de e-mail faz correções ortográficas. D. Programador escreve código num IDE enquanto recebe avisos de análises léxica e sintática.

5. Quando programadores de aplicativos implementam threads, o único recurso do qual eles não podem se beneficiar é
A. O multiprocessamento, pois suas multithreads são vistas como uma única thread pelo kernel. B. A biblioteca de rotinas para criação e gerenciamento delas, fornecida pela linguagem de programação escolhida. C. A troca de contexto entre elas por rotina local, evitando fazer chamadas ao kernel. D. A decisão de como gerenciá-las e sincronizá-las.

6. Considere o modelo de multithreading com as seguintes características: não requer envolvimento do kernel, mas com isso deixa de aproveitar a arquitetura multicore; bloqueia todo o processo caso uma das threads faça uma chamada de E/S. Tal modelo é:
A. Muitos-para-muitos. B. Um-para-muitos C. Um-para-um. D. Muitos-para-um.

7. Considere o modelo de multithreading com as seguintes características: requer envolvimento do kernel, mas com isso aproveita a arquitetura multicore com um número reduzido de threads; não bloqueia o processo caso uma delas faça chamada de E/S. Tal modelo é:

A. Muitos-para-um. B. Um-para-muitos C. Um-para-um. D. Muitos-para-muitos.

8. Quando programadores de sistema implementam threads, a única característica que eles não conseguem obter é
A. Processamento em paralelo de várias delas graças à arquitetura multicore. B. Manutenção de outras threads rodando enquanto alguma delas faz chamada de sistema bloqueante. C. Um SO com kernel mais aliviado de tarefas, uma vez que as threads dividem várias delas entre si. D. Rastreamento adequado de muitas threads por meio de várias tabelas de informações como, por exemplo, registradores e estados de cada uma delas.

9. Dentre as afirmações abaixo, a única incorreta é
A. *Threads* de um processo compartilham recursos e área de memória dele. B. *Threads* de um mesmo processo podem rodar simultaneamente em núcleos de CPU distintos. C. Troca de Contexto ocorre entre processos, mas não entre threads. D. O *multithreading* permite executar um programa mesmo com parte dele bloqueada.

10. Dentre os modelos de multithreading, o único que não existe é
A. Um-para-um. B. Um-para-muitos. C. Muitos-para-muitos. D. Muitos-para-um.

11. O uso de threads melhora o *desempenho* computacional, que consiste de três características: (a) $I$ = intervalo de tempo para a execução de um processo, (b) $Q$ = quantidade de informação processada, (c) $R = I/Q$, razão do tipo "custo-benefício". Usando estes conceitos, podemos dizer que *L*atência e *T*hroughput são, respectivamente:
A. $R$ e $I$. B. $I$ e $1/R$. C. $I$ e $R$. D. $R$ e $1/I$.

12. Dentre os desafios atuais de programar aplicativos beneficiando-se das threads, o único que não se aplica é
A. Kernels de SO são multithreads com gerenciamentos específicos (dispositivos, RAM, interrupções), donde a necessidade de algoritmos de escalonamento de vários núcleos de CPU em paralelo. B. A divisão de um programa em tarefas deve prever que elas sejam realizadas, cada uma com a mesma ordem de esforço computacional. C. Modificar programas e projetos existentes para garantir sincronia de dados e paralelismo de tarefas num mesmo código. D. Testes e depurações precisam ser feitos com mais planejamento.

†

| | |
|---|---|
| UFABC | **Universidade Federal do ABC**<br>**Bacharelado em Ciência da Computação**<br>**Disciplina:** Sistemas Operacionais<br>**Prof.:** VALÉRIO BATISTA<br>**Turma:** DA1 DA2 NA1 NA2 — **Sala:** A-108/114-0<br>**Exame:** SO - Lista6 — **Data:** 22-03-2024 |

**Ass.:** _____

**Estudante:** **ID/RA:**

#E580#V78850 - 2024-03-21 - 20:21:57

| | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|
| A | | | | | | | | | | |
| B | | | | | | | | | | |
| C | | | | | | | | | | |
| D | | | | | | | | | | |

**Instruções:**

(a) Favor repassar suas respostas neste link: https://forms.gle/KUvECwhGzEoM12zJ6
(b) Formulário disponível até 29/3/24 às 23:59:59
(c) Somente a 1a. submissão será registrada (re-submissões serão invalidadas)

## Questões de Múltipla Escolha:

1. Considere o seguinte código em C:

```
#include<stdio.h>
int main(){
 int n,sum=0,m;
 printf("Enter a number: ");
 scanf("%d",&n);
 while(n>0){
  m=n%10;
  sum=sum+m;
  n=n/10;}
 printf("Theosophical addition = %d\n",sum);
 return 0;}
```

Das afirmações abaixo, a única correta é:
A. CPU-burst: 3,7,8,9; IO-burst: 4,5,10. B. CPU-burst: 7,8,9; IO-burst: 4,5,10. C. CPU-burst: 4,5,10; IO-burst: 7,8,9. D. CPU-burst: 7,8,9; IO-burst: 3,4,5,10.

2. Considere os seguintes conceitos: (1) no. de processos completados por unidade de tempo; (2) demora desde a submissão de um processo até sua conclusão; (3) período em que o processo permanece na *Ready Queue*; (4) demora desde a submissão de um processo até vir uma 1a. resposta dele. Tais conceitos se referem a, respectivamente:
A. Turnaround, tempo de espera, tempo de resposta e throughput. B. Tempo de espera, tempo de resposta, throughput e turnaround. C. Throughput, turnaround, tempo de espera e tempo de resposta. D. Tempo de resposta, throughput, turnaround e tempo de espera.

3. As seguintes alternativas se referem a processo preemptivo, exceto:
A. Pode ser interrompido caso outro, mais importante ou urgente, entre na *Ready-Queue* como o próximo da fila. B. Cede "voluntariamente" o controle para outro (por comando exit()), que passa a executar ao invés daquele. C. Roda na CPU até antes dum período pré-estabelecido, depois é suspenso e outro passa a executar. D. Permite um compartilhamento mais balanceado da CPU entre os processos.

4. Seja um escalonamento FCFS com processos $P_1$ a $P_4$, de durações de pico 21, 3, 6 e 2, respectivamente. Se eles chegarem na ordem 1 a 4, o tempo de espera médio será:
A. 25 B. 18.75 C. 8 D. 7.5

5. Seja um escalonamento SJF preemptivo com processos $P_1$ a $P_5$, de durações de pico 6, 2, 8, 3, 4 mas chegando nos momentos 2, 5, 1, 0 e 4, respectivamente. O tempo de espera médio será:
A. 4.6 B. 5.2 C. 3.4 D. 2.2

6. Alguns ítens que caracterizam processos: (1) gastam a maior parte do tempo computando; (2) possuem pequenos picos de uso da CPU; (3) esperam ocasionalmente por eventos de E/S; (4) utilizam pouca CPU. Podemos dizer:
A. CPU-bound: 1,3; IO-bound: 2,4. B. CPU-bound: 3,4; IO-bound: 1,2. C. CPU-bound: 1,2; IO-bound: 3,4. D. CPU-bound: 2,4; IO-bound: 1,3.

7. Considerando a figura sobre Estados de um Processo, temos as seguintes transições: (1) admitido; (2) despacho do escalonador; (3) interrupção; (4) espera por E/S ou evento; (5) término de E/S ou evento; (6) saída.

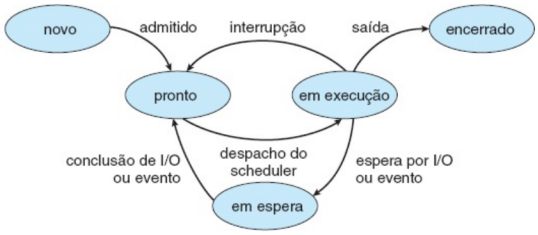


Figura 164: Estados de um Processo.

As circunstâncias em que pode ocorrer escalonamento de CPU são:
A. 3,4,5,6. B. 2,3,4,5. C. 2,3,4,5,6. D. 2,3,4,6.

8. Seja um escalonamento SJF não-preemptivo com processos $P_1$ a $P_5$, de durações de pico 6, 2, 8, 3, 4 mas chegando nos momentos 2, 5, 1, 0 e 4, respectivamente. O tempo de espera médio será:
A. 4.6 B. 5.2 C. 3.4 D. 2.2

9. Seja um escalonamento RR com quantum = 2 e processos $P_1$ a $P_3$, de durações de pico 4, 3 e 5, respectivamente. O tempo de espera médio será:
A. 3.4 B. 5.5 C. 3.7 D. 3.0

10. Tomando como referência a figura sobre Estados de um Processo, e o enunciado da questão referente a ela, considere as seguintes situações de Escalonamento da CPU (ECPU): (a) um processo rodando na CPU pode ser interrompido para que outro passe a ser executado; (b) um processo roda na CPU sem ser interrompido por nenhum evento externo; (c) o ECPU ocorre somente nas situações 4 e 6; (d) o ECPU ocorre em qualquer situação dentre 3 a 6. Podemos dizer:
A. Não-preemptivo: b,c; preemptivo: a,d. B. Não-preemptivo: b,d; preemptivo: a,c. C. Não-preemptivo: a,c; preemptivo: b,d. D. Não-preemptivo: a,d; preemptivo: b,c.

†

| | |
|---|---|
| UFABC | **Universidade Federal do ABC**<br>**Bacharelado em Ciência da Computação**<br>**Disciplina:** Sistemas Operacionais<br>**Prof.:** VALÉRIO BATISTA<br>**Turma:** DA1 DA2 NA1 NA2 **Sala:** A-108/114-0<br>**Exame:** SO - Lista7 **Data:** 29-03-2024 |

**Ass.:** _____

**Estudante:** **ID/RA:**

#E580#V79176 - 2024-03-28 - 20:17:45

| | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| A | | | | | | | | | | | | |
| B | | | | | | | | | | | | |
| C | | | | | | | | | | | | |
| D | | | | | | | | | | | | |

**Instruções:**

(a) Favor repassar suas respostas neste link: https://forms.gle/ueC5YSay4YD81NLDA
(b) Formulário disponível até 5/4/24 às 23:59:59
(c) Somente a 1a. submissão será registrada (re-submissões serão invalidadas)

**Questões de Múltipla Escolha:**

1. Considere os seguintes conceitos envolvendo dois processos P, Q e uma região crítica R: (1) P e Q acessam R concomitantemente; (2) P não consegue ser concluído enquando Q usa algum recuso, e vice-versa; (3) P quer acessar R uma 2a. vez, mas na 1a. deixou o *turn* para Q sem que este se interesse mais por R. De 1-3, tais conceitos se referem a, respectivamente

A. Corrida, Deadlock e Starvation. B. Starvation, Corrida e Deadlock. C. Deadlock, Starvation e Corrida. D. Corrida, Starvation e Deadlock.

2. Dentre os suportes de hardware que ajudam a evitar as situações de corrida, o único que não se aplica é

A. Interrupção do processo. B. Troca de Conteúdo entre duas variáveis. C. Acesso a recurso disponível na RAM. D. Leitura e Atualização de um valor da RAM.

3. Considere um processo com apenas duas threads, consumidora e produtora, sendo o código desta dado abaixo, na qual `BUFFER_SIZE` e `count` são globais:

```
1  void *insert(void *args){
2   int valor =* (int *) args;
3   while (count==BUFFER_SIZE);
4   ++count;
5   buffer[count]=valor;}
```

As linhas da região crítica, como integridade dos dados, são

A. 3,4. B. Apenas a 4. C. 3,4,5. D. 4,5.

4. Quanto à solução de Peterson para exclusão mútua, a única alternativa errada é

A. O algoritmo garante a exclusão mútua para duas tarefas concorrentes. B. O algoritmo é correto para muitas tarefas concorrentes, pois as variáveis $i, j$ assumem quaisquer valores inteiros. C. Logo antes de tentarem acessar a região crítica, duas tarefas concorrentes nunca vão conflitar quanto a qual delas terá o acesso. D. Nenhuma das tarefas espera indefinidamente para ter acesso à região crítica.

5. O problema Produtor-Consumidor é um exemplo de

A. Starvation, mas não de Deadlock. B. Deadlock, mas não de Starvation. C. Sincronismo perfeito (nem Deadlock, nem Starvation). D. Deadlock e Starvation.

6. Dados os processos $P_1$ a $P_n$ com região crítica $RC$, considere os seguintes conceitos: (1) Depois que $P_1$ solicita para entrar na $RC$, há um limite de vezes em que $P_2$ a $P_n$ entram ali antes de $P_1$; (2) Se $P_1$ executa na $RC$,

então ele é o único ali ($P_2$ a $P_n$ estão todos excluídos desta $RC$); (3) Nenhum dentre $P_2$ a $P_n$ pode bloquear $P_1$ indefinidamente, se ele solicitar entrar na $RC$. Os conceitos (1) a (3) referem-se a, respectivamente:

A. Progresso, Espera Limitada e Exclusão Mútua. B. Exclusão Mútua, Progresso e Espera Limitada. C. Exclusão Mútua, Espera Limitada e Progresso. D. Espera Limitada, Exclusão Mútua e Progresso.

7. Com respeito ao mutex, ou semáforo binário $S$, a única alternativa que não se aplica é

A. A função `release` executa $++S$ e em seguida a tarefa sai da região crítica. B. A função `acquire` testa se $S = 1$, e executa $--S$ em caso afirmativo. C. $S$ começa com valor 0 (i.e., nenhuma tarefa está acessando a região crítica). D. Qualquer tarefa precisa chamar `acquire` para entrar na seção crítica e ter exclusividade ali.

8. A única vantagem em desabilitar interrupções com suporte de hardware é

A. Reduzir este mecanismo a uma única CPU. B. Conceder ao processo a prerrogativa de reabilitá-las quando ele encerrar. C. Não impedir que outras CPUs interfiram no que uma CPU esteja executando. D. Dar a um processo a exclusividade de acessar a memória compartilhada.

9. A exclusão mútua por trava

A. Sempre pode falhar. B. Falha caso não coloquemos um "delay" após lermos o valor da trava. C. Falha caso não coloquemos um "delay" antes de lermos o valor da trava. D. Falha quando há mais de duas tarefas concorrentes.

10. Com respeito ao semáforo contador $S$ com $n$ recursos disponíveis, a única alternativa que não se aplica é

A. A função `acquire` testa se $S > 0$, e executa $--S$ em caso afirmativo. B. A função `release` executa $++S$ e em seguida a tarefa sai da região crítica. C. Caso $S == 0$ e alguma tarefa quiser acessar recurso, então é preciso que uma outra chame `release`. D. $S$ começa com valor 0 (i.e., nenhum recurso está sendo usado).

11. Sejam P e C duas threads de um mesmo processo tais que, em linguagem de máquina, atualizam um contador `count` de ítens num buffer da seguinte forma: `register1 = count`, `register1 = register1 + 1` e `count = register1` (etapas P1, P2 e P3, respec.), `register2 = count`, `register2 = register2 - 1` e `count = register2` (etapas C1, C2 e C3, respec.). Uma vez que as threads rodam em paralelo, se as etapas descritas ocorrerem nos seguntes tempos $t_1 < t_2 < t_3 < t_4$, com $t_1$ (P1 e C1), $t_2$ (P2 e C2), $t_3$ (C3) e $t_4$ (P3), o valor de `count` será

A. Um a menos que o de início. B. Um a mais que o de início. C. Igual ao valor de início. D. Indefinido.

12. Com respeito à exclusão mútua por chaveamento, a única alternativa falsa é

A. Pode tornar a região crítica inacessível. B. Se for implementada para três ou mais tarefas concorrentes, duas delas podem acessar a região crítica em concomitância. C. Não precisa de "delay" para lermos o valor de *turn* (nem antes, nem depois). D. Não pode ser implementada para mais que duas tarefas concorrentes.

†

| | |
|---|---|
| UFABC | **Universidade Federal do ABC**<br>**Bacharelado em Ciência da Computação**<br>**Disciplina:** Sistemas Operacionais<br>**Prof.:** VALÉRIO BATISTA<br>**Turma:** DA1 DA2 NA1 NA2 **Sala:** A-108/114-0<br>**Exame:** SO - Lista8 **Data:** 05-04-2024 |

**Ass.:** _____

**Estudante:** **ID/RA:**

#E580#V79579 - 2024-04-04 - 20:28:11

| | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| A | ○ | ○ | ○ | ○ | ○ | ○ | ○ | ○ | ○ | ○ | ○ | ○ |
| B | ○ | ○ | ○ | ○ | ○ | ○ | ○ | ○ | ○ | ○ | ○ | ○ |
| C | ○ | ○ | ○ | ○ | ○ | ○ | ○ | ○ | ○ | ○ | ○ | ○ |
| D | ○ | ○ | ○ | ○ | ○ | ○ | ○ | ○ | ○ | ○ | ○ | ○ |

**Instruções:**

(a) Favor repassar suas respostas neste link: https://forms.gle/S7fkCzfAjpkRKBpUA
(b) Formulário disponível até 12/4/24 às 23:59:59
(c) Somente a 1a. submissão será registrada (re-submissões serão invalidadas)

**Questões de Múltipla Escolha:**

1. A técnica Overlays, de melhoramento de utilização da RAM, possui as seguintes características, exceto:
A. Módulos só ficam na RAM enquanto estiverem em execução. B. Não requer suporte do SO, pois é o programador que planeja os Overlays no código. C. Divisão do programa em módulos que executam sem sujeição e usam área comum da RAM. D. Ideal para processos cujo tamanho é menor do que o espaço disponível na RAM.

2. Em sua estrutura mais simples, o conjunto de endereços $E$ que a CPU vai gerando enquanto executa um processo $P$ é transformado na *Memory Management Unit* (MMU) pelo registrador de realocação, para re-alocar $P$ no espaço de endereços da RAM dado por
A. $n+E$, onde $n$ é algum número inteiro. B. $n+E$, onde $n$ é algum número positivo ou nulo. C. $n+E$, onde $n$ é algum número negativo ou nulo. D. $n+E$, onde $n$ é algum número positivo.

3. Com respeito à figura abaixo, podemos dizer que

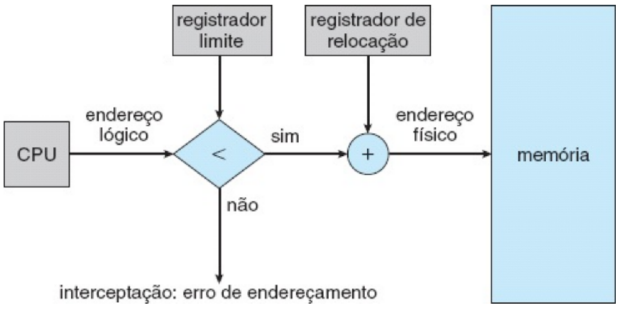


Figura 1: Suporte de Hardware para Realocação.

A. Mostra um procedimento no qual a CPU simplesmente interrompe o processo caso os registradores de realocação e limite tentem violar uma posição ou tamanho indevido na RAM. B. Tal procedimento serve apenas para Partições Dinâmicas, pois estratégias modernas de alocação na RAM nunca interrompem processos. C. É um mecanismo obsoleto, pois hoje em dia os SOs alocam o processo (ou parte dele) no SSD, caso não caiba inteiramente numa partição livre da RAM. D. Esse mecanismo evita a fragmentação interna, mas cria o problema da fragmentação externa.

4. Com respeito à Estratégia de Paginação, para alocar programas ou processos $P$ na RAM, considere os seguintes conceitos: (1) blocos de tamanho fixo que mapeiam toda a RAM; (2) blocos de tamanho pré-fixado pelo total de

†**MCTest:** gerador e corretor de exames disponível para professores - www.ufabc.edu.br

endereços lógicos que $P$ deve ocupar na RAM; (3) associação de cada bloco $i$ (desse 2º tipo) com um único bloco $j$ (daquele 1º tipo). De (1)-(3) tais conceitos são, respectivamente:

A. Quadros, páginas e tabela de quadros. B. Páginas, quadros e tabela de páginas. C. Quadros, páginas e tabela de páginas. D. Páginas, quadros e tabela de quadros.

5. À medida que escrevemos, depuramos e corrigimos um programa, sempre que formos gerando seu executável para verificarmos se ele roda corretamente, teremos as seguintes situações quanto ao seu código assembly na RAM: (1) Não precisará de segmentação para ser alocado; (2) O compilador terá de segmentá-lo para que ele caiba nos espaços livres; (3) O SO terá de gerenciar os espaços que ele ocupa dinamicamente. Os Tempos (1)-(3) referem-se a, respectivamente:

A. Compilação, Carga e Execução. B. Carga, Execução e Compilação. C. Execução, Compilação e Carga. D. Carga, Compilação e Execução.

6. Considerando-se os Endereços *L*ógicos e *F*ísicos, com respeito aos Tempos de *Co*mpilação, *Ca*rga e *Ex*ecução, podemos dizer que:

A. $L = F$ em $Ex$ e $Co$, mas $L \neq F$ em $Ca$. B. $L \neq F$ em $Co$ e $Ca$, mas $L = F$ em $Ex$. C. $L \neq F$ em $Co$, mas $L = F$ em $Ca$ e $Ex$. D. $L = F$ em $Co$ e $Ca$, mas $L \neq F$ em $Ex$.

7. Os ítens abaixo nomeiam estratégias de alocação na RAM, exceto

A. Partições (fixas/dinâmicas). B. Swapping. C. Segmentação. D. Paginação.

8. Considere as seguintes estratégias de alocação de um programa $P$ na RAM, em que $P$ vai para o: (1) 1o. bloco livre, bastando apenas ter tamanho suficiente para que ele seja encaixado; (2) menor bloco livre, em geral o mínimo suficiente para que ele seja encaixado; (3) maior bloco livre, em geral bem mais que o suficiente para que ele seja encaixado. As estratégias (1)-(3) são, respectivamente:

A. First-fit, best-fit e worst-fit. B. Worst-fit, first-fit e best-fit. C. Best-fit, worst-fit e first-fit. D. Best-fit, first-fit e worst-fit.

9. Com respeito à Fila de Entrada, podemos dizer que

A. É o mesmo que Fila de Novos. B. É o mesmo que Fila de Prontos (ou *Ready Queue*). C. Não pode ser Fila de Processos, pois processo é instanciamento de um programa (de modo a habilitá-lo para execução). D. Trata-se de uma fila de processos no HDD (ou SDD), guardada por *swap-in/out*, para liberar espaço na RAM.

10. Com respeito ao Carregamento Dinâmico (CD), não podemos dizer que

A. CD é útil quando temos grande quantidade de código destinado a lidar com eventos freqüentes. B. O emprego do CD gera melhor utilização de espaço da RAM. C. Não requer suporte do SO, pois é o programador que planeja o CD no código. D. No CD carrega-se rotina de programa somente quando ela é chamada.

11. Quando um compilador gera código executável, associa instruções e variáveis em endereços de memória durante as seguintes etapas, exceto:

A. Depuração. B. Carregamento na RAM. C. Execução. D. Tradução em Assembly.

12. Com respeito à Estratégia de Segmentação, para alocar programas ou processos $P$ na RAM, e única alternativa falsa é:

A. Pode causar fragmentação interna. B. A memória lógica não é dividida em páginas. C. Os segmentos de um programa já vem separados pelo compilador (*main, local/global variables, functions, stack, etc.*) D. Baseia-se numa visão de usuário.

†

| | |
|---|---|
| UFABC | **Universidade Federal do ABC**<br>**Bacharelado em Ciência da Computação**<br>**Disciplina:** Sistemas Operacionais<br>**Prof.:** VALÉRIO BATISTA<br>**Turma:** A1 A2 **Sala:** A-108/114-0<br>**Exame:** SO - Lista9 **Data:** 03-04-2025 |

**Ass.:** _____

**Estudante:** ANTHONY DELEON DE ALMEIDA COSTA **ID/RA:** 11202131250

#E594#V96850 - 2025-04-03 - 09:37:52

| | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|
| A | | | | | | | | | | |
| B | | | | | | | | | | |
| C | | | | | | | | | | |
| D | | | | | | | | | | |

**Instruções:**

(a) Favor repassar suas respostas neste link: https://forms.gle/m4PjigEhG2tET8Mi8
(b) Formulário disponível até 16/4/25 às 23:59:59
(c) Somente a 1a. submissão será registrada (re-submissões serão invalidadas)

## Questões de Múltipla Escolha:

1. No caso de alocar $q$ quadros para processos $p_1$ a $p_n$, com respectivos tamanhos $t_1$ a $t_n$, temos as seguintes estratégias, exceto:

A. *Aleatória*: vai alocando quadros a páginas aleatoriamente. B. *Proporcional*: cada $p_k$ recebe a parte inteira de $q \cdot t_k / \sum_{i=1}^{n} t_i$. C. *Igual*: cada $p_k$ recebe a parte inteira de $q / \sum_{i=1}^{n} p_i$. D. *Prioridade*: combina $t_k$ e a prioridade de $p_k$ para saber quantos quadros ele receberá.

2. As seguintes afirmações aplicam-se à Memória Virtual, exceto:

A. Serve para reduzir os problemas de fragmentação no uso da RAM. B. Funciona como uma extensão da RAM. C. Trata-se do espaço da RAM destinado à uma Máquina Virtual. D. Possibilita que apenas parte de um programa no disco rígido seja visto como processo na RAM.

3. Com respeito ao Algoritmo Ótimo, de substituição de páginas, não podemos dizer que

A. É o algoritmo mais usado pelos SOs modernos. B. Requer conhecimento futuro da string de referência. C. Substitui a página que ficará por mais tempo sem utilização. D. Sua taxa de falhas é menor que a de qualquer outro algoritmo de substituição de páginas.

4. Com respeito ao Algoritmo FIFO, de substituição de páginas, podemos dizer que

A. Ter mais páginas disponíveis na RAM pode causar mais falhas de página. B. Ter menos páginas disponíveis na RAM garante mais falhas de página. C. Ter menos páginas disponíveis na RAM garante menos falhas de página. D. Ter mais páginas disponíveis na RAM diminui o número de falhas de página.

5. Quando o SO trata uma interrupção por falha de página com Referência Válida, no caso de inexistência de quadro livre ele segue uma das etapas abaixo, exceto:

A. Executa um algoritmo de troca de páginas, mas com controle estatístico que minimize a freqüência desta operação. B. Escolhe uma página sem uso na RAM e a transfere para o disco rígido. C. Carrega a página do disco rígido para um quadro recém-liberado na RAM. D. Chama a função `abort` para terminar o processo.

6. No caso de Paginação sob Demanda, a única alternativa errada é:

A. Demanda mais operações de E/S do que uma paginação convencional. B. Requer menos espaço na RAM por lidar apenas com as partes essenciais de um processo. C. Pode simplesmente referenciar páginas no disco rígido, e colocá-las em quadros só quando necessário. D. Permite que o número de processos na RAM seja bem maior do que num carregamento convencional.

7. Considere um processo P com páginas 0 a 7, das quais somente 0, 2 e 5 estão alocadas na RAM, a saber, nos quadros 4, 6 e 9, respectiv. Então podemos dizer que

A. O processo tem falha de páginas, e então precisa ser interrompido. B. O processo tem falha de páginas, e então todas elas precisam ser recarregadas. C. Suas páginas 1, 3, 4, 6 e 7 têm bit inválido na Tabela de Páginas. D. Suas páginas 1, 3, 4, 6 e 7 estão nos quadros 5, 7, 8, 10 e 11, respect.

8. No caso do Algoritmo LRU (*Least Recently Used*), de substituição de páginas, a única alternativa errada é

A. Sempre terá desempenho melhor do que o Algoritmo FIFO. B. Pode ser implementado com clock, e assim transfere a página mais antiga para o disco rígido. C. Pode ser implementado com pilha, e assim transfere a página da base para o disco rígido. D. Requer suporte de hardware para ser implementado.

9. No caso de obter-se um quadro livre por substituição de página, considere as seguintes afirmações: (1) não relevar a qual processo pertence a página no quadro a ser liberado; (2) para cada processo existe um número fixo de quadros reservados a ele; (3) é a estratégia mais usada pelos SOs; (4) pode aumentar a taxa de falha de página. Ao classificarmos as afirmações (1)-(4) como característica de Substituição *G*lobal ou *L*ocal teremos, respectivamente:

A. $G, G, L, L$ B. $G, L, G, L$. C. $L, L, G, G$. D. $L, G, L, G$.

10. Quanto ao compartilhamento de páginas pela Memória Virtual, não podemos dizer que:

A. Permite a processos diferentes usarem uma página em comum. B. Inclui bibliotecas do SO. C. Só é possível quando a página compartilhada estiver num quadro. D. Agiliza a criação de processos, por exemplo com a chamada `fork()`.